Проектирование и реализация реляционной базы данных для управления библиотекой

Часть 1: Проектирование базы данных

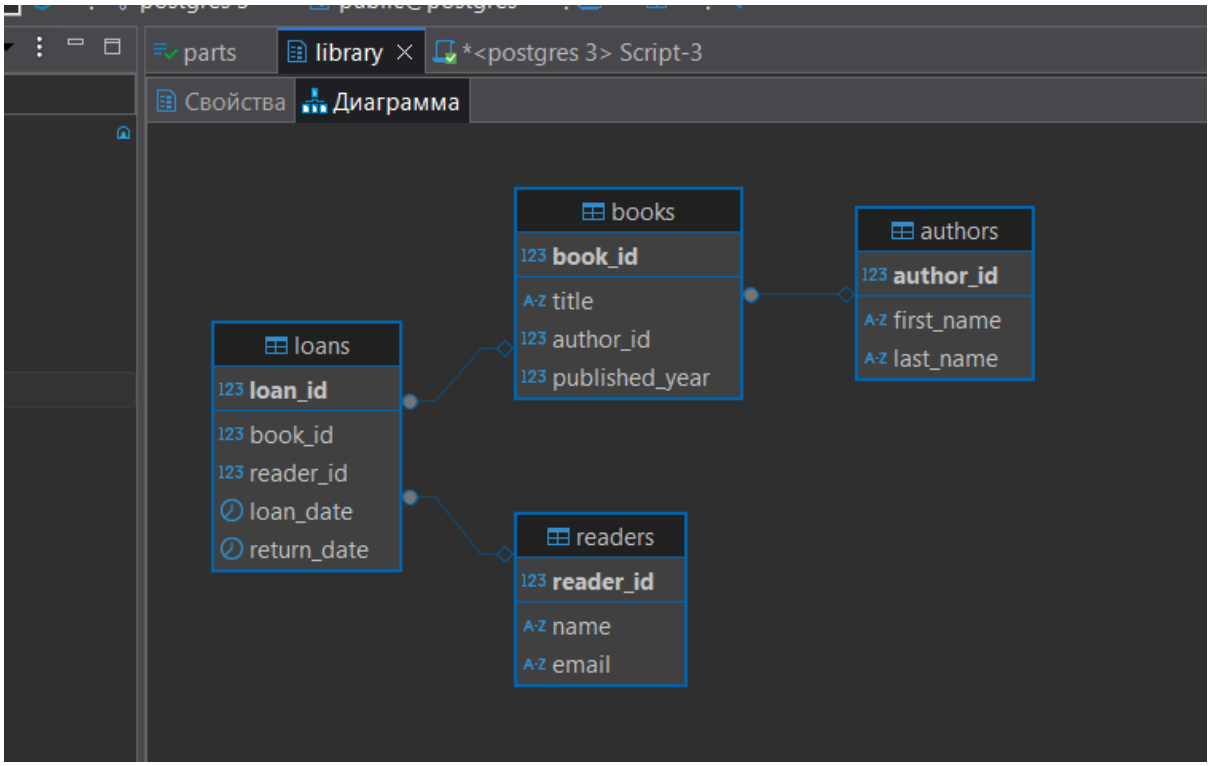
Исходные данные:

Предположим, вам нужно создать реляционную базу данных для управления библиотекой. База данных должна включать следующие сущности:

- Книги (Books)
- Авторы (Authors)
- Читатели (Readers)
- Заказы (Loans)

Задание:

1. Спроектируйте схему базы данных с учетом нормализации до третьей нормальной формы (3NF). Определите атрибуты для каждой сущности и связи между ними.

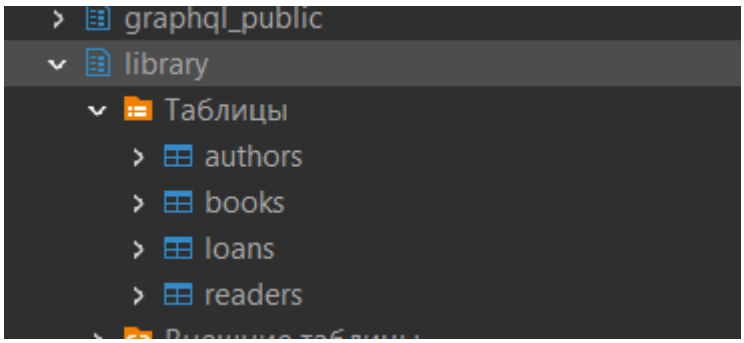


Пример структуры:

- Authors
  - author_id (PK)
  - first_name
  - last_name
- Books
  - book_id (PK)
  - title
  - author_id (FK)
  - published_year
- Readers
  - reader_id (PK)
  - name
  - email
- Loans
  - loan_id (PK)
  - book_id (FK)
  - reader_id (FK)
  - loan_date
  - return_date

Часть 2: Реализация базы данных в PostgreSQL

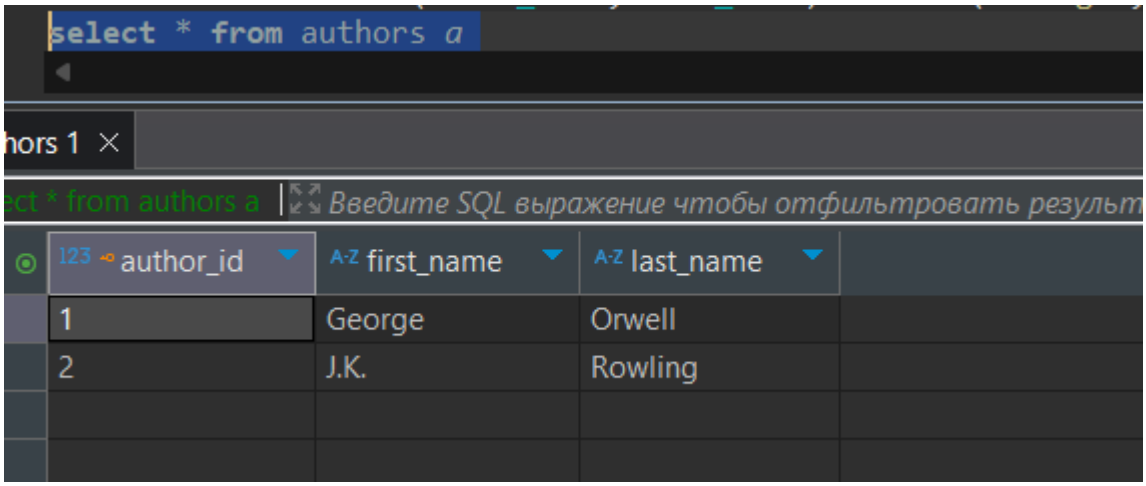
1. Создание таблиц:



```
CREATE TABLE Authors (
   author_id SERIAL PRIMARY KEY,
   first_name VARCHAR(50),
   last_name VARCHAR(50)
);

CREATE TABLE Books (
   book_id SERIAL PRIMARY KEY,
   title VARCHAR(100),
   author_id INT REFERENCES Authors(author_id),
   published_year INT
);

CREATE TABLE Readers (
   reader_id SERIAL PRIMARY KEY,
   name VARCHAR(100),
   email VARCHAR(100) UNIQUE
);

CREATE TABLE Loans (
   loan_id SERIAL PRIMARY KEY,
   book_id INT REFERENCES Books(book_id),
   reader_id INT REFERENCES Readers(reader_id),
   loan_date DATE,
   return_date DATE
);
```

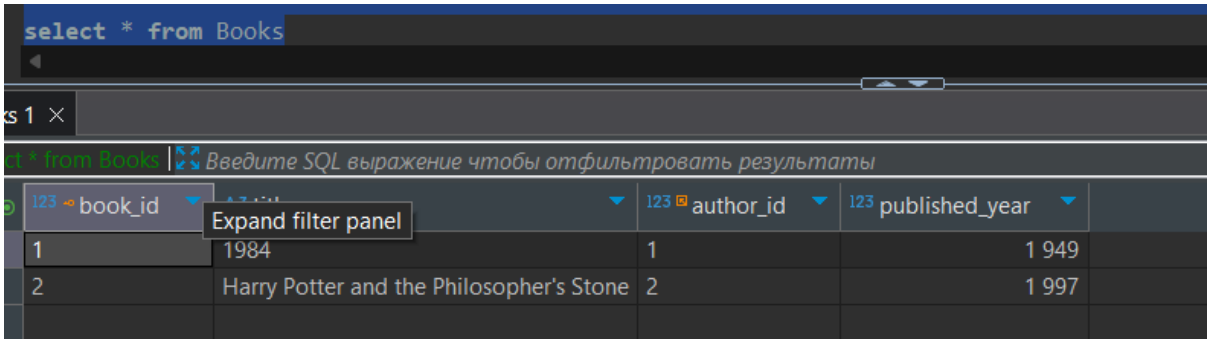
2. Вставка данных (CRUD):

-- Добавление авторов



INSERT INTO Authors (first_name, last_name) VALUES ('George', 'Orwell'), ('J.K.', 'Rowling');

-- Добавление книг

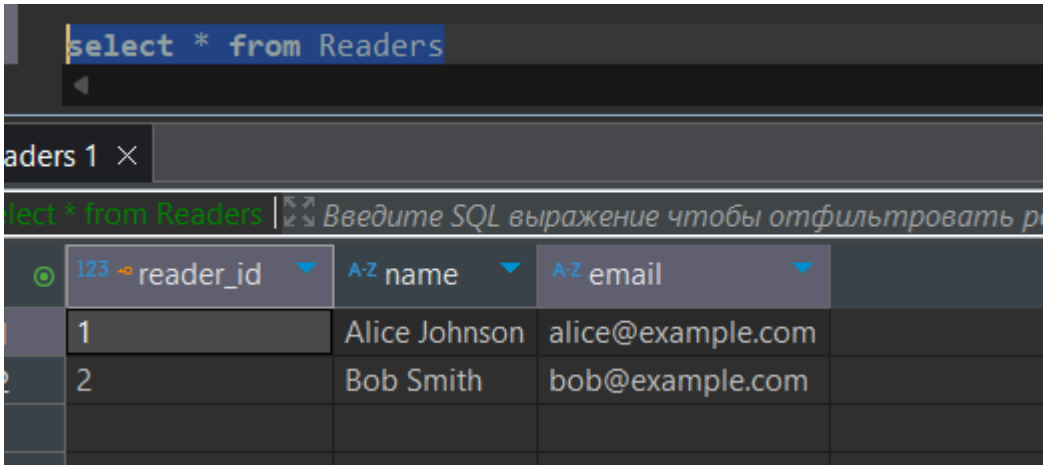


INSERT INTO Books (title, author_id, published_year) VALUES

('1984', 1, 1949),

('Harry Potter and the Philosopher's Stone', 2, 1997);

-- Добавление читателей

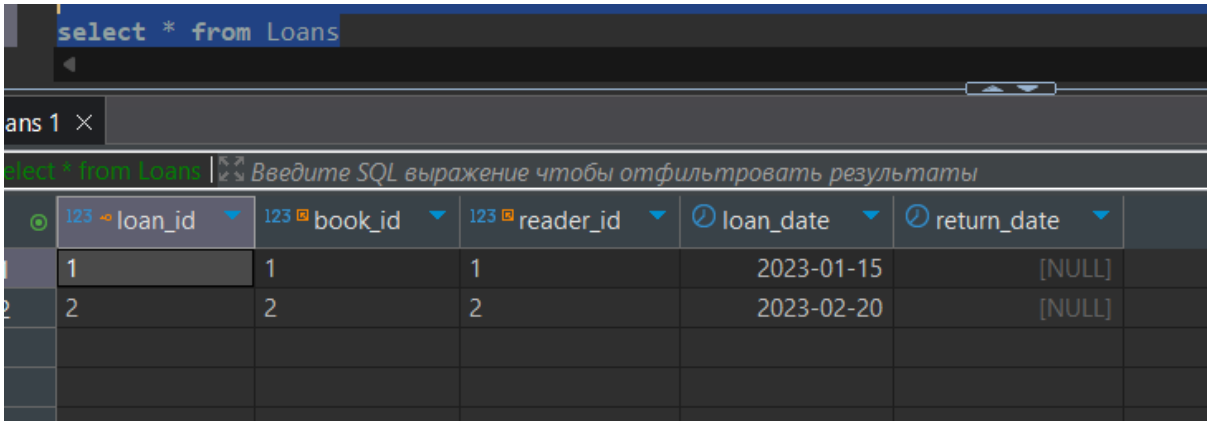


INSERT INTO Readers (name, email) VALUES

('Alice Johnson', 'alice@example.com'),

('Bob Smith', 'bob@example.com');

-- Добавление заказов



INSERT INTO Loans (book_id, reader_id, loan_date) VALUES

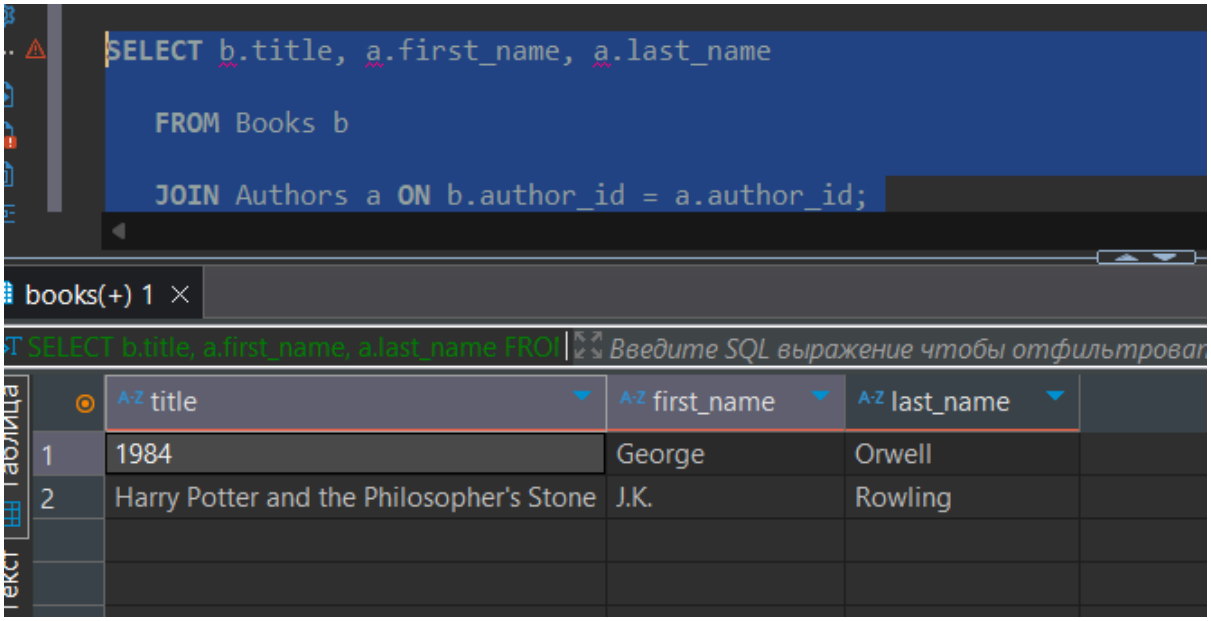
(1, 1, '2023-01-15'),

(2, 2, '2023-02-20');

Часть 3: Запросы и операции

1. Основные операции CRUD:

– Чтение всех книг с информацией об авторах:

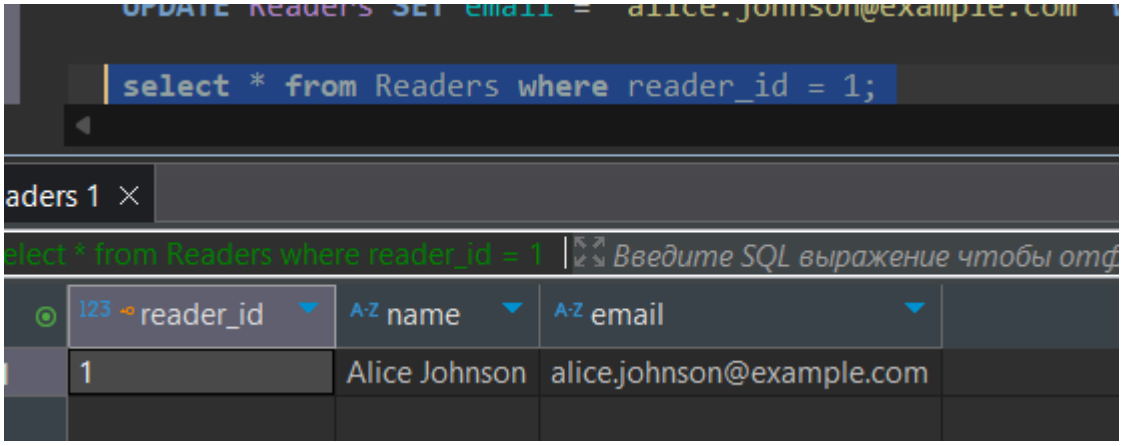


SELECT b.title, a.first_name, a.last_name

FROM Books b

JOIN Authors a ON b.author_id = a.author_id;

• Обновление информации о читателе:



UPDATE Readers SET email = 'alice.johnson@example.com' WHERE reader_id = 1;

• Удаление книги:

```
DELETE FROM Loans WHERE book_id = 1;
DELETE FROM Books WHERE book_id = 1;
SELECT FROM Books WHERE book_id = 1;
```

Результат 1

Введите SQL выражение чтобы отфильтровать результаты

Нет данных

Запустите выполнение запроса (Ctrl+Enter) или скрипта (Alt+X) чтобы увидеть результаты

DELETE FROM Books WHERE book_id = 1;

2. Работа с NULL:

– Выбор всех заказов с указанием даты возврата или значения по умолчанию:

```
SELECT l.loan_id, b.title, COALESCE(l.return_date::TEXT, 'Not returned') AS return_status
FROM Loans l
JOIN Books b ON l.book_id = b.book_id;
```

Введите SQL выражение чтобы отфильтровать результаты

loan_id	title	return_status
2	Harry Potter and the Philosopher's Stone	Not returned

SELECT l.loan_id, b.title, COALESCE(l.return_date, 'Not returned') AS return_status

FROM Loans l

JOIN Books b ON l.book_id = b.book_id;

3. Использование UNION vs UNION ALL:

– Объединение списков читателей и авторов:

```
SELECT name AS person_name FROM Readers
UNION ALL
SELECT CONCAT(first_name, ' ', last_name) AS person_name FROM Authors;
```

результат 1

SELECT name AS person_name FROM Reade | Введите SQL выражение чтобы отфильтровать результат

	person_name
1	Bob Smith
2	Alice Johnson
3	George Orwell
4	J.K. Rowling

SELECT name AS person_name FROM Readers

UNION ALL

SELECT CONCAT(first_name, ' ', last_name) AS person_name FROM Authors;

4. CTE и рекурсивные запросы:

– Создание иерархического запроса для получения всех книг и их авторов:

```
)
SELECT * FROM AuthorBooks;
```

Результат 1

WITH RECURSIVE AuthorBooks AS (SELECT | Введите SQL выражение чтобы отфильтровать результат

	author_id	first_name	last_name	title
1	2	J.K.	Rowling	Harry Potter and the Philosopher's Stone
2	1	George	Orwell	[NULL]

WITH RECURSIVE AuthorBooks AS (

SELECT a.author_id, a.first_name, a.last_name, b.title

FROM Authors a

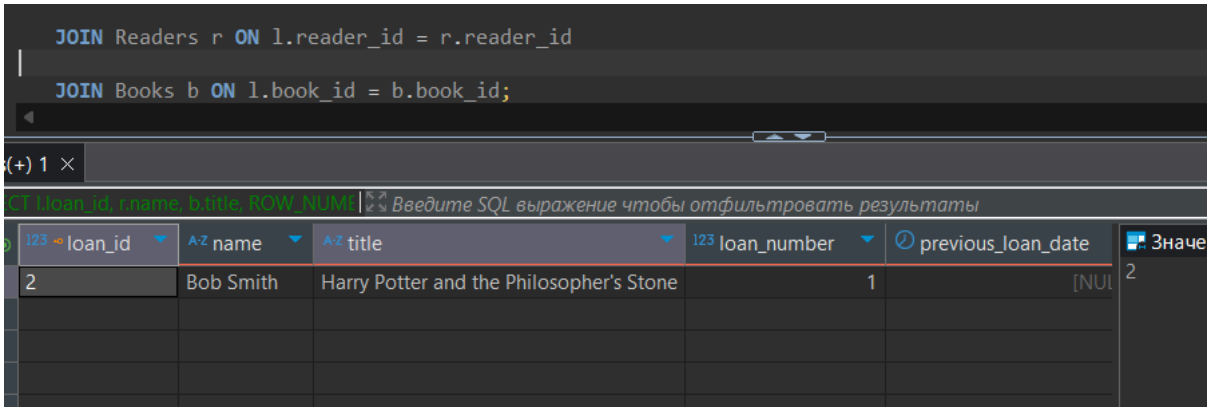
JOIN Books b ON a.author_id = b.author_id

UNION ALL

```
    SELECT a.author_id, a.first_name, a.last_name, NULL AS title
    FROM Authors a
    WHERE NOT EXISTS (SELECT 1 FROM Books b WHERE b.author_id = a.author_id)
)
SELECT * FROM AuthorBooks;
```

5. Оконные функции:

– Использование оконных функций для нумерации заказов:




```
    SELECT l.loan_id, r.name, b.title,
        ROW_NUMBER() OVER (PARTITION BY r.reader_id ORDER BY l.loan_date) AS loan_number,
        LAG(l.loan_date) OVER (PARTITION BY r.reader_id ORDER BY l.loan_date) AS
previous_loan_date
FROM Loans l
JOIN Readers r ON l.reader_id = r.reader_id
JOIN Books b ON l.book_id = b.book_id;
```

Часть 4: Работа с индексами

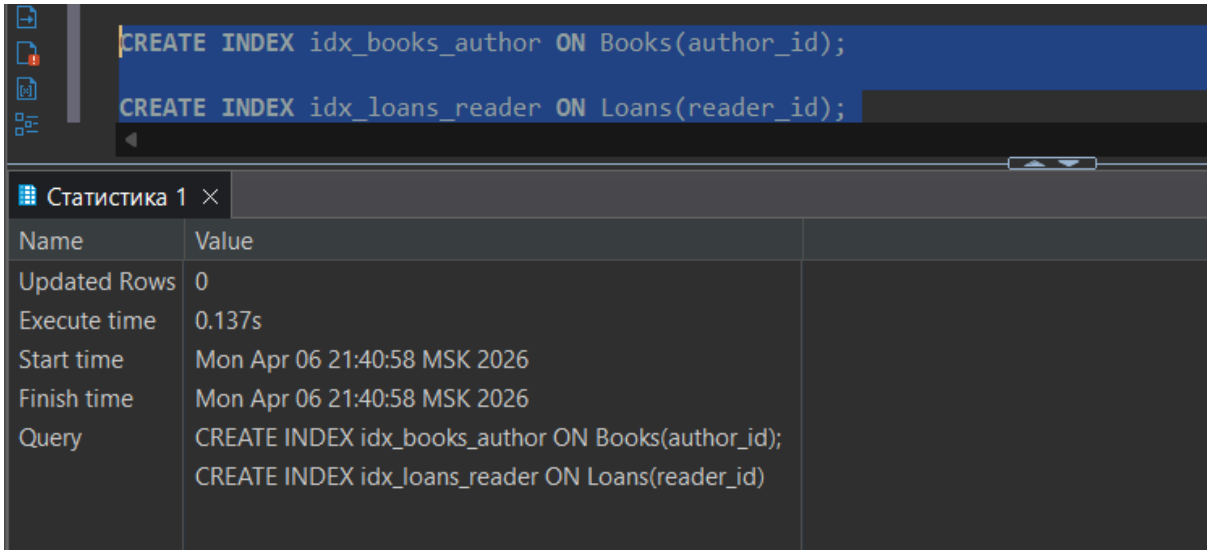
1. Анализ необходимости индексов:

Определите, какие поля часто используются в условиях WHERE и JOIN.

2. Создание индексов:

CREATE INDEX idx_books_author ON Books(author_id);

CREATE INDEX idx_loans_reader ON Loans(reader_id);



Ожидаемые результаты:

• Спроектированная реляционная база данных с нормализованными таблицами.

• Реализованные операции CRUD с примерами SQL-запросов.

• Примеры работы с NULL и объединения данных.

• Реализованы рекурсивные CTE и оконные функции.

• Созданы индексы для оптимизации запросов.